

十大機器學習演算法完整研讀報告

圖文並茂版 - 繁體中文正常字型 PDF

十大機器學習演算法學習地圖

演算法	類型	主要任務	學習重點
線性迴歸	Linear Regression	預測連續數值	用一條最佳直線描述特徵與目標之間的平均關係。
邏輯斯迴歸	Logistic Regression	預測二元或多類別標籤	把線性分數轉換成機率，再用門檻決定類別。
決策樹	Decision Tree	建立可解釋的規則	依照特徵條件逐層切分資料，形成樹狀判斷流程。
隨機森林	Random Forest	提升穩定度與泛化能力	訓練多棵樹並投票或平均，降低單棵樹過度擬合。
支援向量機	Support Vector Machine	建立最大間隔分類器	找出能把類別分開且間隔最大的超平面。
K 近鄰演算法	K-Nearest Neighbors	以相似案例做推論	根據最近的 K 個樣本投票或平均。
朴素貝葉斯	Naive Bayes	文字分類與風險判斷	用貝葉斯定理估計類別機率，並假設特徵條件獨立。
K-Means 分群	K-Means Clustering	把資料分成相似群組	反覆更新群中心與成員，最小化點到中心的距離。
主成分分析	Principal Component Analysis, PCA	壓縮特徵與視覺化	找出資料變異最大的方向，保留主要資訊並降低維度。
神經網路與深度學習	Neural Networks / Deep Learning	學習複雜非線性模式	透過多層節點與參數學習影像、語言與序列中的深層表

本報告以初學者能理解、實務工作者能查閱為目標，系統整理十種常見機器學習演算法。每章皆包含核心概念、運作流程、應用案例、優缺點、資料準備、評估方法與學習建議。文中所有圖表皆為重新製作的示意圖，未使用使用者提供的參考圖。

目錄

1. 線性迴歸 (Linear Regression)
 2. 邏輯斯迴歸 (Logistic Regression)
 3. 決策樹 (Decision Tree)
 4. 隨機森林 (Random Forest)
 5. 支援向量機 (Support Vector Machine)
 6. K 近鄰演算法 (K-Nearest Neighbors)
 7. 樸素貝葉斯 (Naive Bayes)
 8. K-Means 分群 (K-Means Clustering)
 9. 主成分分析 (Principal Component Analysis, PCA)
 10. 神經網路與深度學習 (Neural Networks / Deep Learning)
- 附錄：演算法選擇指南、評估指標與學習路線

第 1 章 線性迴歸 (Linear Regression)

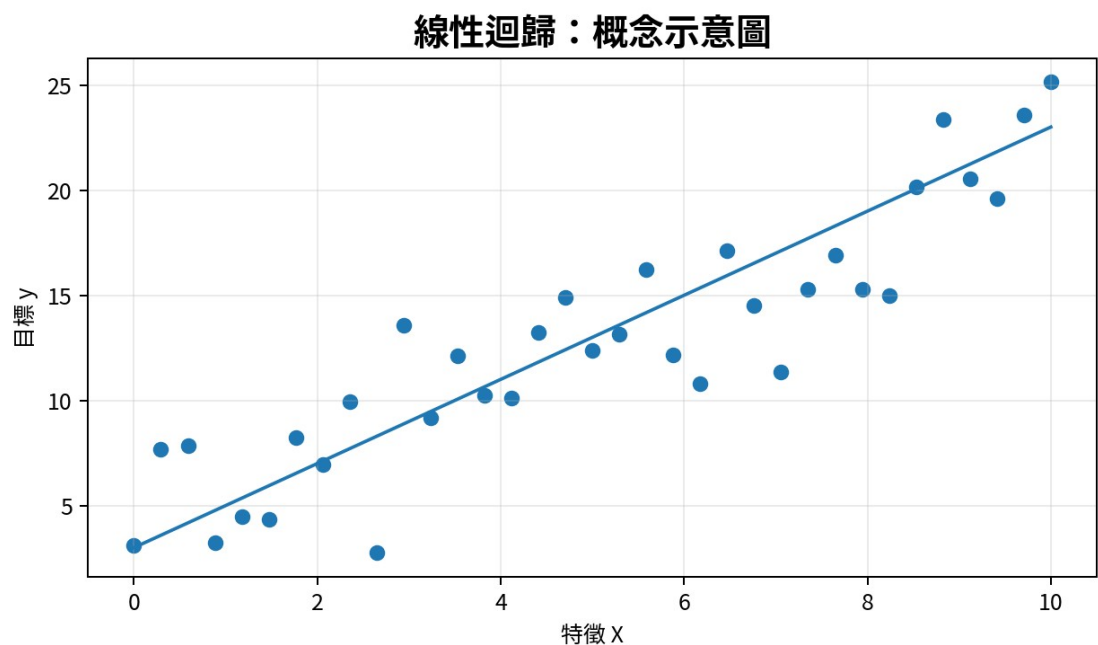


圖 1：線性迴歸 的核心概念示意。此圖為本報告重新繪製，用於輔助理解演算法的運作方式。

模型類型	監督式學習 - 回歸
主要用途	預測連續數值
核心想法	用一條最佳直線描述特徵與目標之間的平均關係。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

線性迴歸適合用來預測連續型數值，例如房價、營收、用電量、考試分數或廣告轉換金額。它假設目標值可以由一組特徵加權相加後得到，因此模型輸出容易解釋。係數為正代表特徵增加時預測值傾向上升，係數為負則代表相反方向。

常見例子是利用坪數、屋齡、捷運距離與樓層預測房價。訓練時模型會尋找一組係數，使預測值與真實值之間的平方誤差總和最小。實務上需要檢查離群值，因為極端交易價格可能大幅拉動迴歸線。

線性迴歸的優點是快速、透明、容易作為基準模型。缺點是難以捕捉複雜非線性關係，也容易受到共線性影響。若特徵尺度差異很大，雖然普通最小平方方法仍可計算，但在正則化迴歸中通常要先標準化。

二、運作流程

線性迴歸 的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第五，

使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用 線性迴歸，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習 線性迴歸 時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用 Python 與 scikit-learn 實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，線性迴歸 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 2：在真實專案中，線性迴歸 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 3：在真實專案中，線性迴歸 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 4：在真實專案中，線性迴歸 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

第 2 章 邏輯斯迴歸 (Logistic Regression)

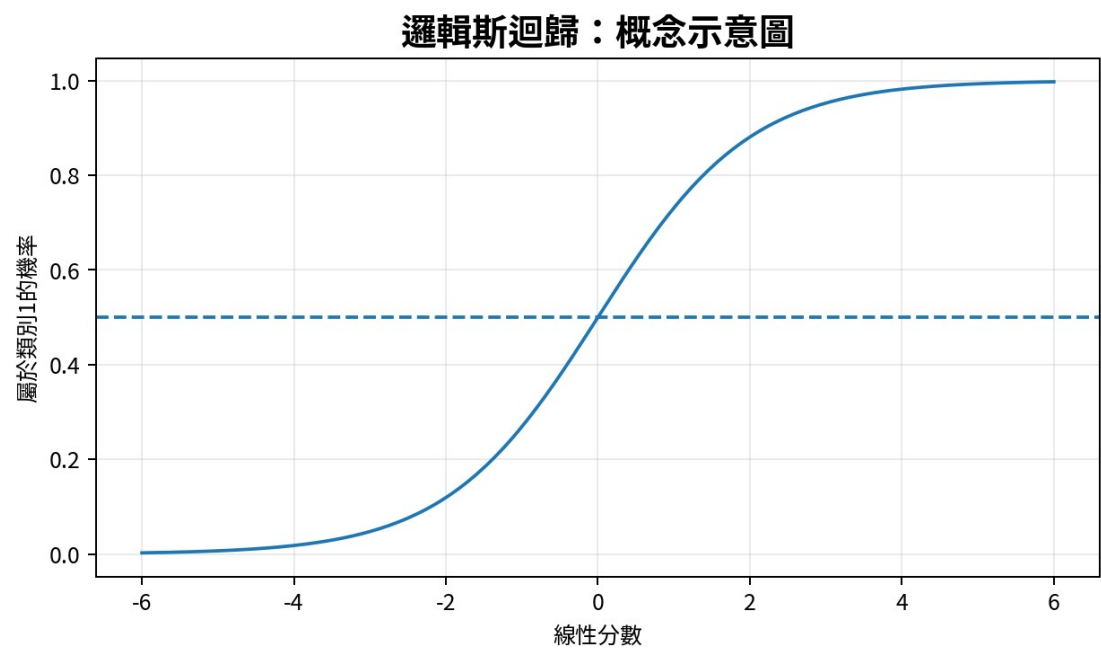


圖 2：邏輯斯迴歸 的核心概念示意。此圖為本報告重新繪製，用於輔助理解演算法的運作方式。

模型類型	監督式學習 - 分類
主要用途	預測二元或多類別標籤
核心想法	把線性分數轉換成機率，再用門檻決定類別。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

邏輯斯迴歸雖然名稱含有迴歸，但主要用於分類。它先計算線性分數，再透過 sigmoid 函數轉換成 0 到 1 之間的機率。當機率超過門檻，例如 0.5，就判定為正類。

常見例子包括判斷客戶是否流失、交易是否詐欺、病患是否高風險。模型係數可解讀為特徵對勝算的影響，因此在金融、醫療與商業決策中很受歡迎。

它的優點是訓練快、可解釋、能輸出機率。限制是決策邊界主要是線性的，若類別分布高度非線性，需要加入交互項、多項式特徵或改用其他模型。

二、運作流程

邏輯斯迴歸 的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第

五，使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用 邏輯斯迴歸，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習 邏輯斯迴歸 時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用 Python 與 scikit-learn 實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，邏輯斯迴歸 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 2：在真實專案中，邏輯斯迴歸 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 3：在真實專案中，邏輯斯迴歸 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 4：在真實專案中，邏輯斯迴歸 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

第 3 章 決策樹 (Decision Tree)

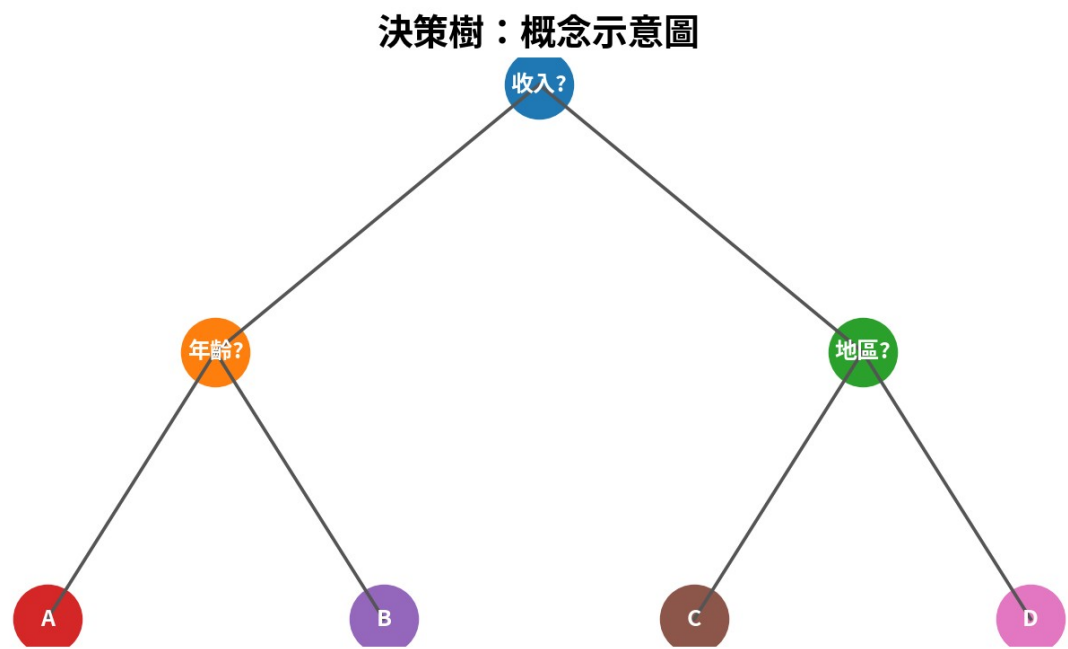


圖 3：決策樹 的核心概念示意。此圖為本報告重新繪製，用於輔助理解演算法的運作方式。

模型類型	監督式學習 - 分類/回歸
主要用途	建立可解釋的規則
核心想法	依照特徵條件逐層切分資料，形成樹狀判斷流程。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

決策樹用一連串條件把資料切成較純的區塊，例如「收入是否大於五萬元」、「是否曾購買」、「年齡是否小於三十」。每個節點都代表一個問題，每條分支代表問題的答案。

在信用審核中，決策樹可以把申請者依照收入、負債比、信用紀錄分流，最後得到通過或拒絕的預測。因為規則像流程圖，非技術人員也能理解。

決策樹容易過度擬合，尤其樹太深時會記住訓練資料的細節。常用控制方式包含限制最大深度、最小葉節點樣本數與剪枝。

二、運作流程

決策樹 的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第五，

使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用 決策樹，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習 決策樹 時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用 Python 與 scikit-learn 實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，決策樹 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

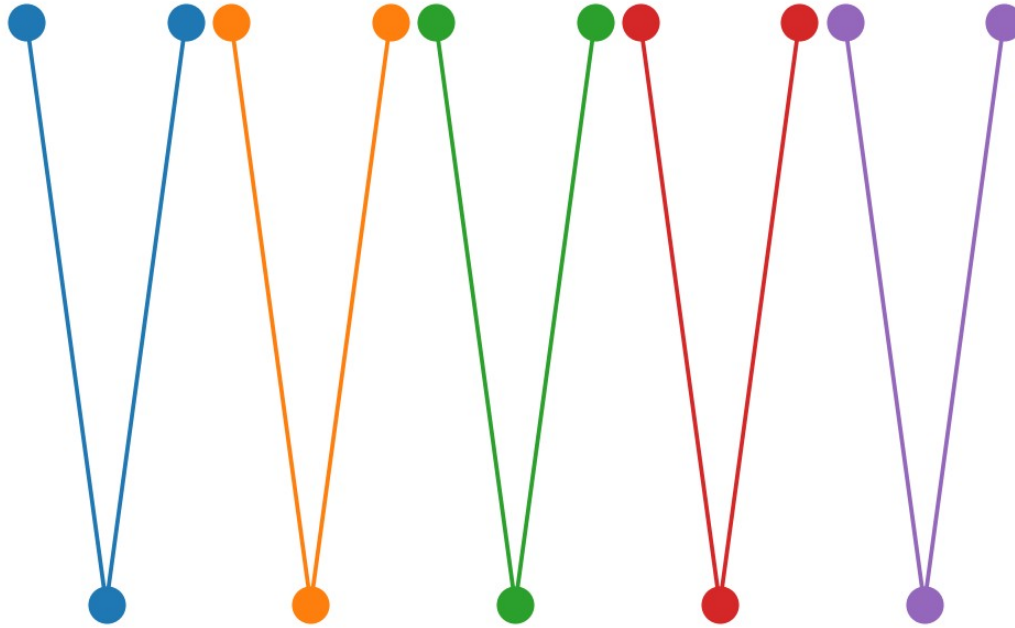
延伸閱讀重點 2：在真實專案中，決策樹 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 3：在真實專案中，決策樹 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 4：在真實專案中，決策樹 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

第 4 章 隨機森林 (Random Forest)

隨機森林：概念示意圖



多棵樹投票 / 平均 → 較穩定的預測

圖 4：隨機森林 的核心概念示意。此圖為本報告重新繪製，用於輔助理解演算法的運作方式。

模型類型	監督式學習 - 集成模型
主要用途	提升穩定度與泛化能力
核心想法	訓練多棵樹並投票或平均，降低單棵樹過度擬合。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

隨機森林是由許多決策樹組成的集成模型。每棵樹會看到不同的抽樣資料與部分特徵，最後透過投票或平均產生結果。這種隨機性讓模型比單棵樹更穩定。

在客戶分群後的流失預測、保險理賠風險、製造良率預測中，隨機森林常能提供強健表現。它也能估計特徵重要度，幫助分析哪些因素對預測最有貢獻。

缺點是模型體積較大，推論速度比單棵樹慢，解釋性也較差。若資料量非常大或需要即時預測，需要留意效能。

二、運作流程

隨機森林的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第五，使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用隨機森林，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免

把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習 隨機森林 時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用 Python 與 scikit-learn 實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，隨機森林 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 2：在真實專案中，隨機森林 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 3：在真實專案中，隨機森林 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 4：在真實專案中，隨機森林 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

第 5 章 支援向量機 (Support Vector Machine)

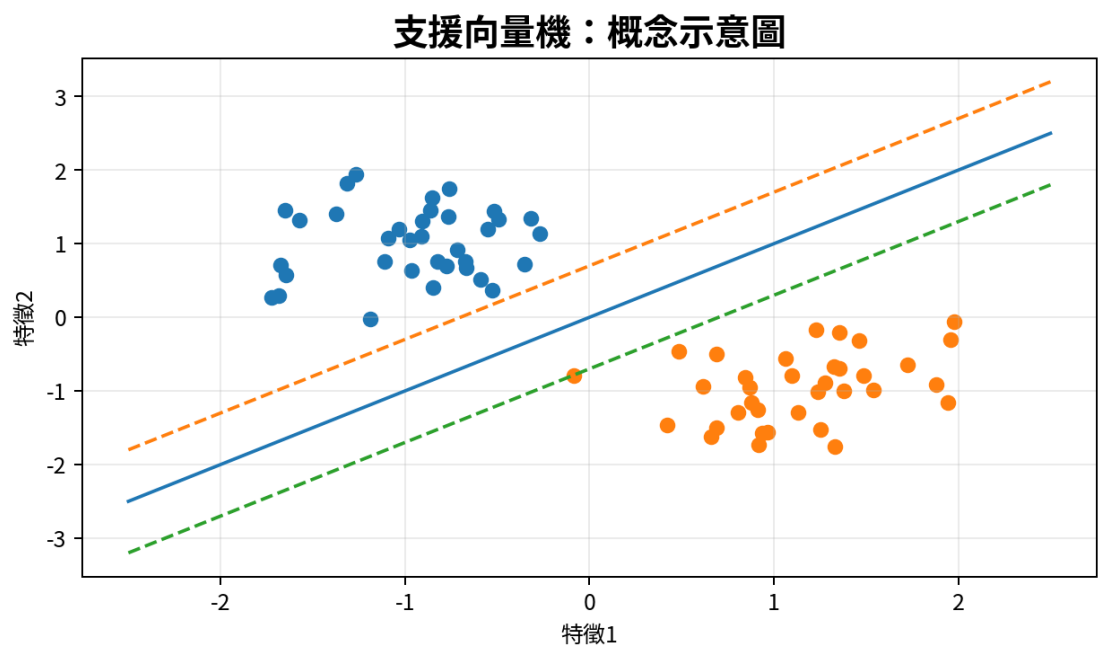


圖 5：支援向量機 的核心概念示意。此圖為本報告重新繪製，用於輔助理解演算法的運作方式。

模型類型	監督式學習 - 分類/回歸
主要用途	建立最大間隔分類器
核心想法	找出能把類別分開且間隔最大的超平面。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

支援向量機的核心是最大間隔。它尋找一條分隔線或超平面，使不同類別之間的距離最大。靠近邊界的少數資料點稱為支援向量，會決定模型的分隔位置。

在文字分類、影像特徵分類、小樣本高維度資料中，SVM 常有不錯表現。透過核函數，它可以把資料映射到更高維空間，處理非線性分類問題。

SVM 對特徵縮放很敏感，通常需要標準化。當資料量非常大時，訓練成本可能偏高，參數 C 與 gamma 也需要調整。

二、運作流程

支援向量機 的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第

五，使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用 支援向量機，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習 支援向量機 時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用 Python 與 scikit-learn 實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，支援向量機 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 2：在真實專案中，支援向量機 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 3：在真實專案中，支援向量機 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 4：在真實專案中，支援向量機 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

第 6 章 K 近鄰演算法 (K-Nearest Neighbors)

K 近鄰演算法：概念示意圖

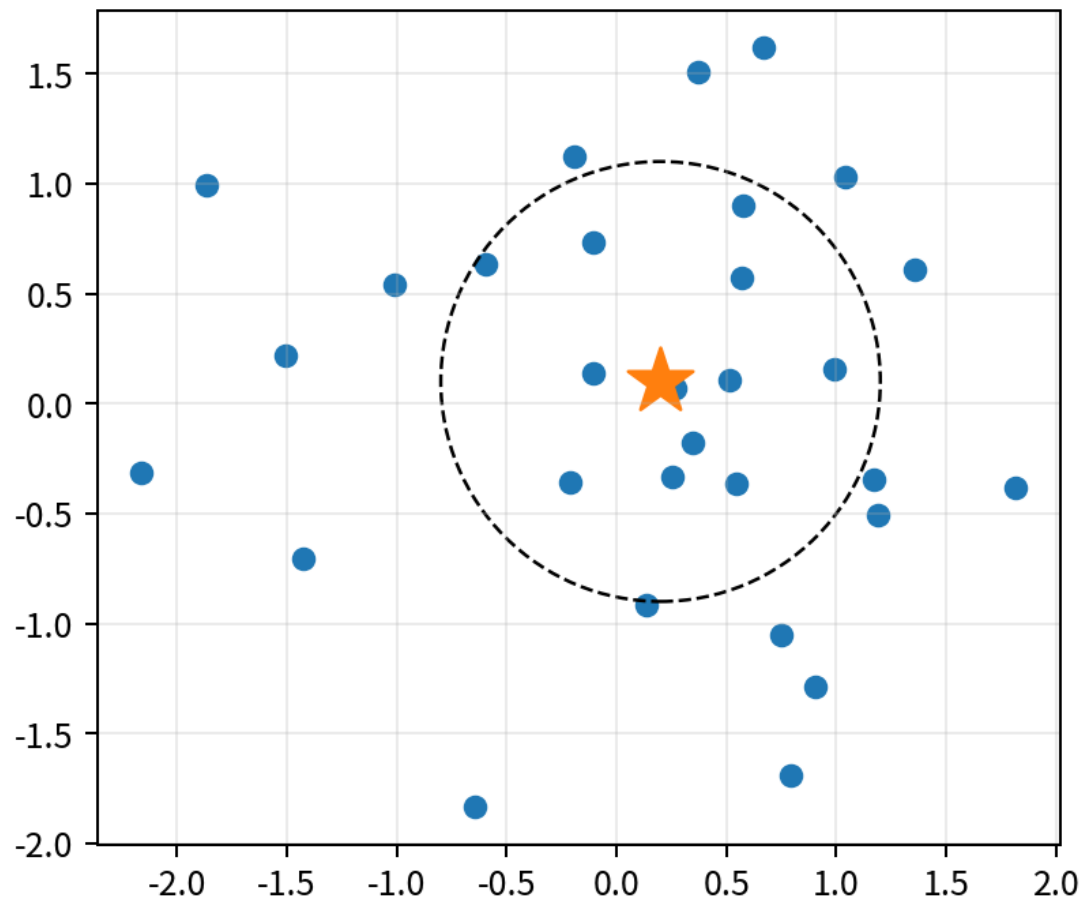


圖 6：K 近鄰演算法 的核心概念示意。此圖為本報告重新繪製，用於輔助理解演算法的運作方式。

模型類型	監督式學習 - 分類/回歸
主要用途	以相似案例做推論
核心想法	根據最近的 K 個樣本投票或平均。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

KNN 是非常直覺的演算法：新資料的類別由附近 K 個鄰居決定。若用於回歸，則取鄰居目標值的平均。它幾乎沒有明確訓練階段，重點在於距離計算。

例如要預測一位新顧客是否會購買，可以找出歷史資料中與他年齡、收入、瀏覽行為最相似的 K 位顧客，再依多數決預測。

KNN 容易理解，但在高維度資料中距離可能失去意義，也會受到特徵尺度影響。因此通常要標準化，並選擇合適的 K 值與距離指標。

二、運作流程

K 近鄰演算法 的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第五，使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用 K 近鄰演算法，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習 K 近鄰演算法 時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用 Python 與 scikit-learn 實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，K 近鄰演算法 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能

快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 2：在真實專案中，K 近鄰演算法 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 3：在真實專案中，K 近鄰演算法 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 4：在真實專案中，K 近鄰演算法 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

第 7 章 樸素貝葉斯 (Naive Bayes)

樸素貝葉斯：概念示意圖

$$P(\text{類別} \mid \text{特徵}) \propto P(\text{特徵} \mid \text{類別}) \times P(\text{類別})$$

常用於垃圾信、情緒、文件主題分類

圖 7：樸素貝葉斯 的核心概念示意。此圖為本報告重新繪製，用於輔助理解演算法的運作方式。

模型類型	監督式學習 - 分類
主要用途	文字分類與風險判斷
核心想法	用貝葉斯定理估計類別機率，並假設特徵條件獨立。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

樸素貝葉斯根據貝葉斯定理估計類別機率，並假設在給定類別後，各特徵彼此獨立。這個假設在真實世界往往不完全成立，但模型仍常有良好表現。

典型應用是垃圾郵件分類。模型會根據文字中出現的詞彙，如「免費」、「中獎」、「發票」等，估計郵件屬於垃圾郵件或正常郵件的機率。

它的優點是速度快、需要資料量相對少、適合高維稀疏文字資料。缺點是條件獨立假設較強，若特徵高度相關，機率估計可能不準。

二、運作流程

樸素貝葉斯 的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第

五，使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用 樸素貝葉斯，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習 樸素貝葉斯 時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用 Python 與 scikit-learn 實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，樸素貝葉斯 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 2：在真實專案中，樸素貝葉斯 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 3：在真實專案中，樸素貝葉斯的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸研讀重點 4：在真實專案中，樸素貝葉斯的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

第 8 章 K-Means 分群 (K-Means Clustering)

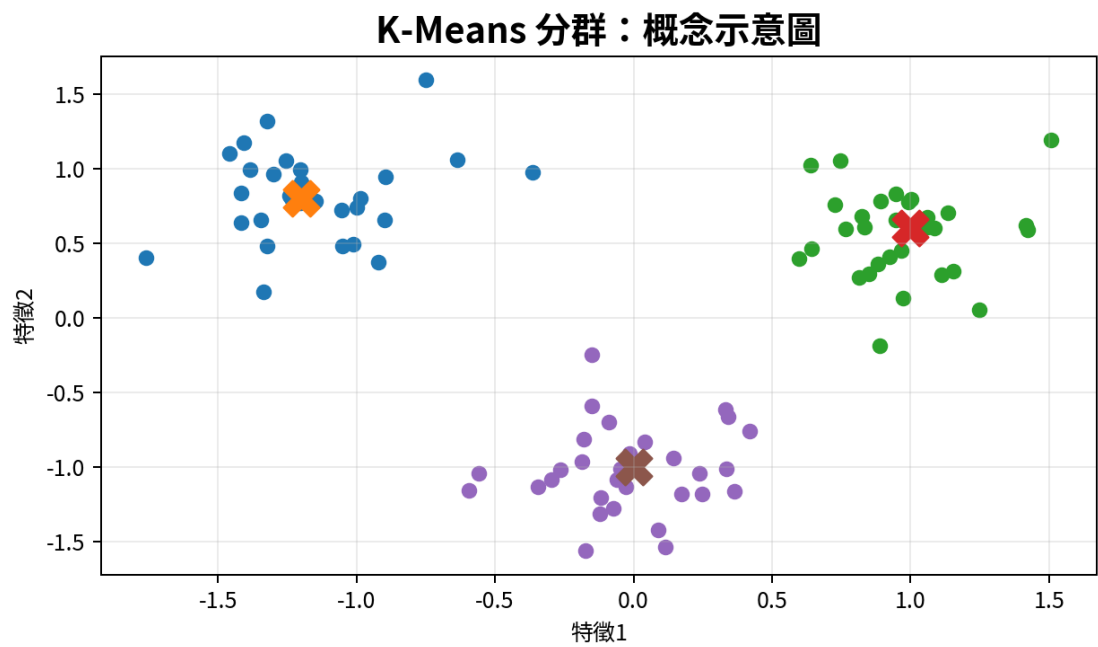


圖 8：K-Means 分群 的核心概念示意。此圖為本報告重新繪製，用於輔助理解演算法的運作方式。

模型類型	非監督式學習 - 分群
主要用途	把資料分成相似群組
核心想法	反覆更新群中心與成員，最小化點到中心的距離。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

K-Means 是非監督式學習演算法，目標是在沒有標籤的情況下，把資料分成 K 個群。模型會先設定群中心，接著反覆把資料分配給最近中心，並更新中心位置。

在會員經營中，可以用消費頻率、平均客單價、最近一次購買時間等特徵把顧客分群，進而設計不同促銷策略。

K-Means 需要事先決定 K 值，且對離群值與初始中心敏感。常用肘部法或輪廓係數協助選擇群數。

二、運作流程

K-Means 分群的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第五，使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用 K-Means 分群，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習 K-Means 分群時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用 Python 與 scikit-learn 實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，K-Means 分群的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 2：在真實專案中，K-Means 分群的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 3：在真實專案中，K-Means 分群的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能

快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 4：在真實專案中，K-Means 分群的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

第 9 章 主成分分析 (Principal Component Analysis, PCA)

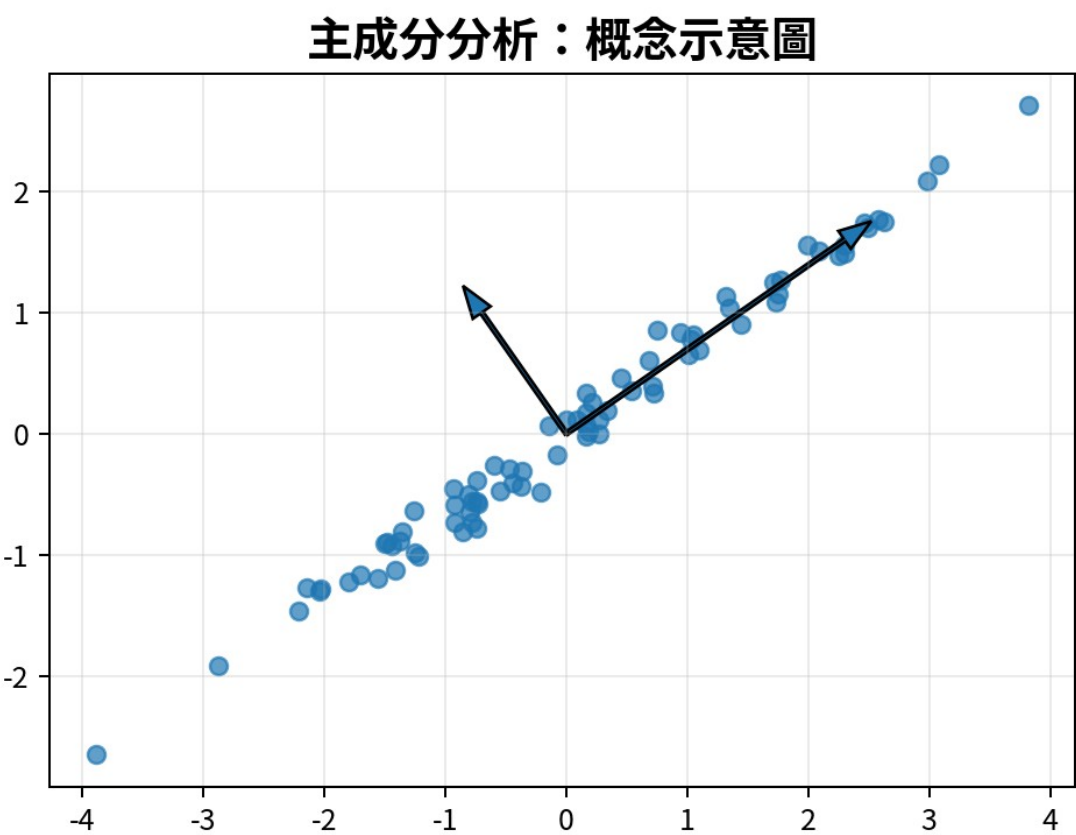


圖 9：主成分分析 的核心概念示意。此圖為本報告重新繪製，用於輔理解演算法的運作方式。

模型類型	非監督式學習 - 降維
主要用途	壓縮特徵與視覺化
核心理念	找出資料變異最大的方向，保留主要資訊並降低維度。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

PCA 用於降維，目標是把多個相關特徵轉換成較少的主成分。第一主成分保留最多變異，第二主成分在與第一主成分垂直的限制下保留次多變異。

例如有大量財務指標、問卷題項或影像像素時，PCA 可以壓縮資料、降低噪音、方便視覺化，也能減少後續模型訓練成本。

PCA 的限制是主成分是線性組合，解釋性可能不如原始特徵直覺。使用前通常需要標準化，否則尺度大的特徵會主導結果。

二、運作流程

主成分分析 的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第五，使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用 主成分分析，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習 主成分分析 時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用 Python 與 scikit-learn 實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，主成分分析 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快

速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 2：在真實專案中，主成分分析的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 3：在真實專案中，主成分分析的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 4：在真實專案中，主成分分析的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

第 10 章 神經網路與深度學習 (Neural Networks / Deep Learning)



圖 10：神經網路與深度學習 的核心概念示意。此圖為本報告重新繪製，用於輔助理解演算法的運作方式。

模型類型	監督式/非監督式/自監督式
主要用途	學習複雜非線性模式
核心想法	透過多層節點與參數學習影像、語言與序列中的深層表徵。
適合資料	結構化表格資料、文字特徵、影像特徵或經過工程化處理的數值資料，視演算法而定。

一、核心概念

神經網路由多層節點組成，每個節點會接收輸入、加權、加上偏差，再經過非線性啟動函數。多層結構讓模型能學習複雜模式。

在影像辨識、語音辨識、自然語言處理、推薦系統與生成式 AI 中，深度學習已成為核心技術。卷積網路適合影像，循環網路與 Transformer 適合序列與語言。

深度學習需要較多資料、計算資源與調參經驗。若資料量不足，容易過度擬合；若模型過大，部署成本也會提高。

二、運作流程

神經網路與深度學習 的典型流程可以分成五個步驟。第一，確認問題型態：是預測數值、分類標籤、尋找群組，還是降低維度。第二，整理資料欄位，將不能直接計算的資訊轉換成模型可使用的特徵。第

三，切分訓練集、驗證集與測試集，避免在同一份資料上同時調參與評估。第四，訓練模型並調整關鍵參數。第五，使用合適指標評估結果，並觀察錯誤案例。對初學者而言，最重要的是把演算法視為一套假設，而不是萬能工具；資料是否符合假設，往往比演算法名稱更重要。

三、實務例子

以商業資料分析為例，企業可能擁有顧客年齡、地區、消費次數、平均客單價、最近一次互動時間、客服紀錄與活動點擊紀錄。若使用神經網路與深度學習，可以根據任務設計不同目標：預測下月消費金額、判斷是否流失、找出相似顧客群、或壓縮大量特徵以利視覺化。建模時要先定義成功標準，例如預測誤差要低於某一金額、召回率要高於某一比例，或分群後每群都能對應到具體行銷策略。

四、優點、限制與常見錯誤

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

實作時建議先建立簡單基準模型，再逐步增加複雜度。資料前處理包含缺失值補補、異常值檢查、類別變數編碼、特徵縮放與資料切分。評估模型時不能只看訓練集分數，必須使用驗證集或交叉驗證，避免把偶然噪音誤認為真正規律。若模型用於商業決策，還要評估錯誤成本，例如誤判高風險客戶與漏判高風險客戶的代價可能完全不同。模型完成後也不是結束，仍需要監控資料漂移、定期重訓、記錄版本，並確認預測結果能被使用者理解。

五、學習建議

學習神經網路與深度學習時，不建議只背公式。較有效的方法是先用小型資料集手算或畫圖理解，再用Python與scikit-learn實作，最後觀察參數改變對結果的影響。建議保留實驗紀錄，包括資料版本、特徵欄位、參數設定、評估指標與錯誤分析。當你能說明「為什麼這個模型適合目前問題」以及「模型錯在哪些案例」，才算真正掌握。

延伸閱讀重點 1：在真實專案中，神經網路與深度學習的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 2：在真實專案中，神經網路與深度學習的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後

仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 3：在真實專案中，神經網路與深度學習 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

延伸閱讀重點 4：在真實專案中，神經網路與深度學習 的表現通常受資料品質影響甚深。若資料來源不穩定、欄位定義前後不一致、標籤有延遲或人工標註標準不一，模型即使在測試集上分數不錯，部署後仍可能快速失效。因此，建議將資料字典、特徵生成邏輯、模型版本與評估報告一起保存。對商業團隊而言，模型的價值不只是準確率，而是能否讓決策更快、更一致、成本更低，並且能在錯誤發生時追溯原因。

附錄：演算法選擇指南

若目標是預測連續數值，可先從線性迴歸開始，再嘗試隨機森林、梯度提升樹或神經網路。若目標是分類，可先使用邏輯斯迴歸建立可解釋基準，再依資料量與非線性程度選擇決策樹、隨機森林、SVM、KNN 或深度學習。

若沒有標籤，通常先考慮 K-Means、階層式分群或密度式分群；若特徵太多、視覺化困難或模型訓練成本過高，可以考慮 PCA。模型選擇不是一次決定，而是迭代過程。

評估指標要配合任務。回歸常用 MAE、MSE、RMSE 與 R^2 ；分類常用 Accuracy、Precision、Recall、F1-score 與 ROC-AUC；分群可用輪廓係數、群內平方和與商業可解釋性。

附錄二：從零開始的實作流程與專案檢查表

一個完整的機器學習專案，通常不是從選模型開始，而是從定義問題開始。首先要釐清預測結果將被誰使用、在什麼時間點使用、錯誤會造成什麼成本，以及模型是否真的能改變決策。若預測結果無法導向行動，即使模型分數很高，也很難產生實際價值。

資料蒐集階段要確認資料來源、更新頻率、欄位定義、資料權限與隱私要求。許多專案失敗不是因為演算法不好，而是因為訓練資料與部署資料不一致。例如訓練時使用了未來才會知道的欄位，部署時卻無法取得，這種情況稱為資料洩漏。

資料清理階段要處理缺失值、重複值、錯誤值與極端值。缺失值不能只靠平均數填補，還要思考缺失本身是否代表某種訊號。類別欄位也要注意新類別問題，因為部署後可能出現訓練期間沒有看過的商品、地區或客戶類型。

特徵工程是把原始資料轉換成模型可學習訊號的過程。好的特徵可以讓簡單模型表現很好；差的特徵則會讓複雜模型也難以學到穩定規律。常見方法包含標準化、對數轉換、分箱、交互特徵、時間窗口統計、文字向量化與影像特徵抽取。

模型訓練階段要建立基準線。基準線可以是平均值、多數類別、簡單線性模型或舊系統規則。若新模型只比基準線好一點，卻增加大量維護成本，未必值得部署。相反地，若簡單模型已能滿足需求，就不需要一開始追求最複雜的深度學習。

模型評估要區分離線指標與線上效果。離線指標來自測試集，方便快捷比較模型；線上效果則來自實際使用情境，例如轉換率、客服成本、錯誤警示量或使用者滿意度。兩者可能不完全一致，因此重要模型需要 A/B 測試或灰度發布。

部署後要持續監控模型。資料分布可能因季節、活動、競爭環境或使用者行為改變而漂移。若輸入特徵分布明顯改變，模型分數可能下降。成熟團隊會建立監控儀表板，追蹤輸入資料、預測分布、實際標籤、延遲時間與錯誤案例。

模型治理也很重要。每次訓練都應記錄資料版本、程式版本、參數、評估結果與負責人。若模型影響金融、醫療或人事決策，還需要更嚴格的可解釋性、公平性與審核流程。機器學習不是只寫程式，而是一套可追蹤、可驗證、可維護的決策系統。

附錄三：十種演算法的快速比較

線性迴歸適合建立第一個回歸基準；邏輯斯迴歸適合建立第一個分類基準；決策樹適合需要規則解釋的情境；隨機森林適合追求穩定表現並能接受較低解釋性的表格資料；SVM 適合中小型高維資料；KNN 適合理解相似度概念與小型資料；樸素貝葉斯適合文字分類；K-Means 適合探索未知群組；PCA 適合降維與視覺化；深度學習適合大量資料與複雜模式。

初學者常犯的錯誤是直接套用演算法而忽略資料切分。若在清理、標準化或特徵選擇時不小心使用了測試集資訊，模型評估就會過度樂觀。正確做法是把資料處理流程與模型一起放進 pipeline，確保每次交叉驗證都只用訓練折學習轉換規則。

另一個常見錯誤是只看準確率。當資料類別不平衡時，準確率可能具有誤導性。例如詐欺交易只有百分之一，模型全部預測為正常也有百分之九十九準確率，但完全沒有偵測能力。這時應看召回率、精確率、F1 分數、PR 曲線與業務成本。

在簡報或報告中呈現模型結果時，建議同時提供三類資訊：第一是模型分數，第二是錯誤案例，第三是可行建議。只呈現分數容易讓讀者不知道下一步該做什麼；加入錯誤案例與建議，才能讓模型分析轉化成決策。

學習順序建議為：先理解監督式學習與非監督式學習的差異，再學線性迴歸與邏輯斯迴歸建立基礎；接著學 KNN、決策樹與隨機森林理解非線性與集成；再學樸素貝葉斯與 SVM；最後學 K-Means、PCA 與神經網路。這樣的路徑由簡到難，也能逐步建立數學直覺。

線性迴歸適合建立第一個回歸基準；邏輯斯迴歸適合建立第一個分類基準；決策樹適合需要規則解釋的情境；隨機森林適合追求穩定表現並能接受較低解釋性的表格資料；SVM 適合中小型高維資料；KNN 適合理解相似度概念與小型資料；樸素貝葉斯適合文字分類；K-Means 適合探索未知群組；PCA 適合降維與視覺化；深度學習適合大量資料與複雜模式。

初學者常犯的錯誤是直接套用演算法而忽略資料切分。若在清理、標準化或特徵選擇時不小心使用了測試集資訊，模型評估就會過度樂觀。正確做法是把資料處理流程與模型一起放進 pipeline，確保每次交叉驗證都只用訓練折學習轉換規則。

另一個常見錯誤是只看準確率。當資料類別不平衡時，準確率可能具有誤導性。例如詐欺交易只有百分之一，模型全部預測為正常也有百分之九十九準確率，但完全沒有偵測能力。這時應看召回率、精確率、F1 分數、PR 曲線與業務成本。

在簡報或報告中呈現模型結果時，建議同時提供三類資訊：第一是模型分數，第二是錯誤案例，第三是可行建議。只呈現分數容易讓讀者不知道下一步該做什麼；加入錯誤案例與建議，才能讓模型分析轉化成決策。

學習順序建議為：先理解監督式學習與非監督式學習的差異，再學線性迴歸與邏輯斯迴歸建立基礎；接著學 KNN、決策樹與隨機森林理解非線性與集成；再學樸素貝葉斯與 SVM；最後學 K-Means、PCA 與神經網路。這樣的路徑由簡到難，也能逐步建立數學直覺。

附錄四：閱讀本報告的建議方法

閱讀時可以先把每一章分成三個問題：這個演算法想解決什麼問題、它如何從資料中學習、它在什麼情況下容易失敗。只要能回答這三個問題，就能避免把演算法當成黑盒工具。對初學者來說，最有效的方式是每學完一個演算法，就找一份小資料集實作一次，並刻意改變資料比例、特徵數量與參數設定，觀察結果如何變化。

建議建立一份個人模型筆記表，欄位包含任務類型、輸入資料、輸出結果、重要參數、需要的前處理、常用評估指標、優點、缺點與適合案例。當未來遇到新問題時，可以先查表選出三個候選模型，再用相同資料切分與相同評估方式比較。這樣不只學得比較快，也能讓模型選擇更有根據。

若目標是求職或面試，除了能說出演算法定義，更要能說明專案流程。例如：為什麼選這個模型、如何處理缺失值、如何避免資料洩漏、如何解讀混淆矩陣、如何向非技術主管說明模型限制。面試官通常不只想確認你會呼叫函式，也想知道你是否理解資料、指標與商業情境。

若目標是實務導入，請把模型視為產品的一部分。資料管線、權限控管、監控機制、模型更新頻率、使用者介面與回饋流程都會影響成敗。好的模型若無法被使用者信任，仍然很難產生價值；普通模型若能穩定提供清楚建議，反而可能更適合第一版系統。

閱讀時可以先把每一章分成三個問題：這個演算法想解決什麼問題、它如何從資料中學習、它在什麼情況下容易失敗。只要能回答這三個問題，就能避免把演算法當成黑盒工具。對初學者來說，最有效的方式是每學完一個演算法，就找一份小資料集實作一次，並刻意改變資料比例、特徵數量與參數設定，觀察結果如何變化。

建議建立一份個人模型筆記表，欄位包含任務類型、輸入資料、輸出結果、重要參數、需要的前處理、常用評估指標、優點、缺點與適合案例。當未來遇到新問題時，可以先查表選出三個候選模型，再用相同資料切分與相同評估方式比較。這樣不只學得比較快，也能讓模型選擇更有根據。

若目標是求職或面試，除了能說出演算法定義，更要能說明專案流程。例如：為什麼選這個模型、如何處理缺失值、如何避免資料洩漏、如何解讀混淆矩陣、如何向非技術主管說明模型限制。面試官通常不只想確認你會呼叫函式，也想知道你是否理解資料、指標與商業情境。

若目標是實務導入，請把模型視為產品的一部分。資料管線、權限控管、監控機制、模型更新頻率、使用者介面與回饋流程都會影響成敗。好的模型若無法被使用者信任，仍然很難產生價值；普通模型若能穩定提供清楚建議，反而可能更適合第一版系統。